

Documentation Technique Frontend

Vue 3 · TypeScript · Vite · Pinia · Vuetify 3

Version: 1.0
Date: 26 March 2026
Statut: **Production**

Documentation Technique Complète
Plateforme QHSE LOGIQUALI

Architecture Frontend LOGIQUALI

Architecture Frontend LOGIQUALI

- Framework: Vue 3 (Composition API)
- Langage: TypeScript
- Build: Vite
- State Management: Pinia
- UI: Vuetify 3 + Tailwind CSS 4
- Routing: Vue Router (3 modules isolés)
- HTTP: Axios + Intercepteurs
- WebSocket: Laravel Echo + Pusher
- Tests: Vitest + Cypress

Stack : Vue 3 · TypeScript · Vite · Pinia · Vuetify 3 · Tailwind CSS 4

1. Stack technique

Composant	Technologie	Version
Framework	Vue 3	^3.5.21
Langage	TypeScript	~5.9.2
Build	Vite	^7.1.5
State management	Pinia	^3.0.3
UI principale	Vuetify	^3.10.1
CSS utilitaire	Tailwind CSS	^4.1.18
Routing	Vue Router	^4.5.1
HTTP	Axios	^1.13.2
WebSocket	Laravel Echo + Pusher	^2.3.0
Graphiques	ApexCharts + Chart.js + ECharts	—
Calendrier	FullCalendar	^6.1.20
Éditeur riche	Tiptap + Quill	^3.18.0
i18n	vue-i18n	^9.14.5
Tests unitaires	Vitest	^4.0.17
Tests E2E	Cypress	^15.9.0

2. Architecture des sources



3. Architecture modulaire

Le frontend est organisé en 3 modules isolés avec leurs propres routes, composants et composables.

3.1 Module ClientA (entreprise)

Point d'entrée : `modules/clienta/router/index.ts`

Pages disponibles :

Section	Pages
Dashboard	<code>dashboard.vue</code>
Contexte	<code>context/</code> , <code>stakeholders/</code>
Leadership	<code>leadership/</code> (Policy, OrgChart, JobDescriptions)
Planification	<code>planning/</code> (RisksOpportunities, Objectives, ActionPlans)
Support	<code>documents/</code> , <code>competences/</code> , <code>communications/</code> , <code>operations/</code>
Performance	<code>audits/</code> , <code>indicators/</code> , <code>management-reviews/</code>
Amélioration	<code>nonconformities/</code> , <code>actions/</code> , <code>improvement/</code>

ISO	<code>iso/</code> (securite, environnement, energie)
Administration	<code>users/</code> , <code>roles/</code> , <code>sites/</code> , <code>settings/</code> , <code>subscriptions/</code>
Profil	<code>profile/</code>

Composables métier (`modules/clienta/composables/`) :

Composable	Responsabilité
<code>useRisks</code>	CRUD risques, matrice, export
<code>useDocuments</code>	CRUD documents, versioning, approbation
<code>useAudits</code>	Planification, exécution, clôture audits
<code>useActions</code>	Suivi actions, Gantt, Kanban
<code>useNonConformities</code>	Déclaration, analyse, clôture NC
<code>useProcesses</code>	Gestion processus, diagramme tortue
<code>useIndicators</code>	KPI, saisie valeurs, graphiques
<code>useSubscription</code>	Abonnement, modules accessibles
<code>useSites</code>	Gestion sites, sélecteur de site
<code>useUsers</code>	Gestion utilisateurs, rôles
<code>useFormations</code>	Plan de formation, compétences
<code>useCommunications</code>	Plan de communication
<code>useModuleAccess</code>	Vérification accès modules
<code>useSubscribedModules</code>	Modules souscrits actifs

3.2 Module ClientB (client final)

Pages :

- `dashboard.vue` — tableau de bord client
- `ComplaintManagement.vue` — gestion réclamations
- `SatisfactionForms.vue` / `SatisfactionSurveys.vue` — formulaires satisfaction
- `Profile.vue` — profil client
- `Support.vue` — tickets support

3.3 Module SuperAdmin

Pages :

- `dashboard.vue` — statistiques plateforme
- `companies/` — validation entreprises (KYC)
- `norms/` — gestion référentiels normatifs
- `offers/` — gestion offres commerciales

- `subscriptions/` — gestion abonnements
- `users/` — gestion utilisateurs plateforme
- `settings.vue` — paramètres système

4. Stores Pinia

4.1 Stores globaux

Store	Fichier	Responsabilité
Auth	<code>auth.ts</code>	Utilisateur connecté, token, permissions
App	<code>app.ts</code>	État global application
Site Context	<code>siteContext.ts</code>	Site actif sélectionné
Global Loader	<code>globalLoader.ts</code>	État chargement global
Lock Screen	<code>lockScreen.ts</code>	Verrouillage écran

4.2 Stores domaine

Store	Responsabilité
<code>riskStore / improvement/riskStore</code>	Risques, matrice
<code>auditStore / auditProgramStore</code>	Audits, programmes
<code>documentStore</code>	Documents, versions
<code>nonConformityStore</code>	Non-conformités
<code>actionStore</code>	Actions correctives/préventives
<code>objectiveStore</code>	Objectifs QHSE
<code>indicatorStore / indicateurStore</code>	Indicateurs KPI
<code>processStore</code>	Processus
<code>stakeholderStore</code>	Parties intéressées
<code>contextStore</code>	Contexte organisme
<code>managementReviewStore</code>	Revue de direction
<code>siteStore</code>	Sites
<code>userStore</code>	Utilisateurs
<code>subscriptionStore</code>	Abonnements
<code>competenceStore</code>	Compétences, habilitations
<code>habilitationStore</code>	Habilitations ISO 45001
<code>consommationEnergieStore</code>	Consommations ISO 50001
<code>aspectEnvironnementalStore</code>	Aspects ISO 14001

5. Composants UI

5.1 Composants communs (`components/common/`)

Composant	Usage
<code>AppButton</code>	Bouton standardisé
<code>AppModal</code>	Modale générique
<code>AppTable</code>	Tableau avec tri/pagination
<code>AppInput</code> / <code>AppSelect</code> / <code>AppTextarea</code>	Champs formulaire
<code>AppCard</code>	Carte contenu
<code>AppBadge</code> / <code>StatusBadge</code>	Badges statut
<code>AppEmptyState</code>	État vide
<code>AppSpinner</code>	Indicateur chargement
<code>AppToast</code>	Notifications toast
<code>AppProgressBar</code>	Barre progression
<code>AppWidget</code> / <code>AppWidgetCompact</code>	Widgets tableau de bord

5.2 Composants UI primitifs (`components/ui/`)

Basés sur Radix Vue + Headless UI :

- Dialog, DropdownMenu, Tabs, Select, Checkbox, Switch
- Table (TableBody, TableRow, TableCell...)
- Badge, Button, Card, Input, Label, Textarea
- `UniversalDialog`, `UniversalFilters`, `UniversalPagination`, `UniversalSkeleton`
- `GlobalSearch`, `LoadingStates`, `ProgressiveLoader`

5.3 Composants domaine QHSE

Domaine	Composants clés
Actions	<code>ActionCard</code> , <code>ActionForm</code> , <code>ActionGanttView</code> , <code>ActionKanbanView</code> , <code>ActionWizardDialog</code>
Audits	<code>AuditCard</code> , <code>AuditChecklistTab</code> , <code>AuditCalendar</code> , <code>AuditWizardDialog</code> , <code>AuditFindingsTab</code>
Documents	<code>DocumentCard</code> , <code>DocumentPyramidTree</code> , <code>DocumentQuillEditor</code> , <code>ReviewWorkflow</code> , <code>DocumentPreview</code> , <code>VersionHistory</code>

Risques	RiskHeatmapInteractive (composant interactif matrice)
Non-conformités	NCCard, NCCauseAnalysis, NCFilters
Indicateurs	IndicatorChart, IndicatorDashboard, IndicatorTrendWidget
Processus	TurtleDiagramEditor, TurtleCell (diagramme tortue)
Contexte	ContextWizard, PestelRadar, SwotMatrix
Compétences	CompetenceMatrix, GapAnalysisDialog, HabilitationForm
Revue direction	ManagementReviewWizard, ReviewTimeline
Satisfaction	SatisfactionDashboard
Support	PlanMaintenance, InventaireEquipements, MaintenanceAlertes

6. Routing

6.1 Structure des routes

Diagram illustrating a sequence of modules in a client-server architecture:

- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientA (client initial)
- Module ClientB (client final)

The 10th module is also labeled `/clientb/*`.

6.2 Guards de navigation (`router/guards.ts`)

- Vérification authentification (**requiresAuth**)
- Redirection selon **user_type** (super_admin → /superadmin, company → /company, clientb → /clientb)
- **subscriptionGuard** — bloque accès si abonnement inactif
- Gestion erreurs import dynamique (retry automatique)

7. Composables globaux

Composable	Fichier	Responsabilité
<code>useAuth</code>	<code>composables/useAuth.ts</code>	Authentification, déconnexion

<code>usePermissions</code>	<code>composables/usePermissions.ts</code>	Vérification permissions frontend
<code>useFilters</code>	<code>composables/useFilters.ts</code>	Filtres réutilisables
<code>usePagination</code>	<code>composables/usePagination.ts</code>	Pagination générique
<code>useGlobalSearch</code>	<code>composables/useGlobalSearch.ts</code>	Recherche globale
<code>useLoading</code>	<code>composables/useLoading.ts</code>	États de chargement
<code>useContextLoading</code>	<code>composables/useContextLoading.ts</code>	Chargement contextuel
<code>useSnackbar</code>	<code>composables/useSnackbar.ts</code>	Notifications snackbar
<code>useToast</code>	<code>composables/useToast.ts</code>	Notifications toast
<code>useWebSocket</code>	<code>composables/useWebSocket.ts</code>	Connexion WebSocket (Reverb)
<code>useMenu</code>	<code>composables/useMenu.ts</code>	Navigation sidebar

Composables partagés (`modules/shared/composables/`) :

- `useApi` — wrapper Axios avec gestion erreurs
- `useAsyncAction` — action async avec état loading/error
- `useAsyncData` — chargement données async
- `useDebounce` — debounce réactif
- `useTheme` — thème clair/sombre

8. Services frontend

Service	Responsabilité
<code>authService</code>	Login, logout, refresh token
<code>riskService</code> / <code>improvement/riskService</code>	API risques
<code>auditService</code>	API audits
<code>documentService</code>	API documents, upload, download
<code>nonConformityService</code>	API non-conformités
<code>actionService</code>	API actions
<code>indicatorService</code>	API indicateurs KPI
<code>processService</code>	API processus
<code>subscriptionService</code>	API abonnements
<code>userService</code>	API utilisateurs

siteService	API sites
pdfExportService	Export PDF côté client
dashboardService	API tableau de bord
leadershipService	API politique, organigramme
competenceService	API compétences, matrice
habilitationService	API habilitations
satisfactionSurveyService	API enquêtes satisfaction
complaintService	API réclamations
superAdminService	API superadmin
normService	API référentiels normatifs
geoCatalogService	Pays, villes
qrCodeService	Vérification QR codes

9. Internationalisation

Fichiers : `src/i18n/locales/fr.json` et `en.json`

Langues supportées : Français (défaut), Anglais.

Composant de sélection : `components/ui/LanguageSwitcher.vue`

10. Design system

10.1 Tokens CSS (`styles/design-tokens.css`)

Variables CSS pour couleurs, espacements, typographie, ombres.

10.2 Thème Vuetify (`config/theme.ts`)

Couleurs primaires, secondaires, états (success, warning, error, info).

10.3 Tailwind CSS 4

Configuration : `tailwind.config.js` — classes utilitaires pour layouts et espacements.

10.4 Composants ClientA (`modules/clienta/assets/clienta-theme.css`)

Thème spécifique module entreprise.

11. Tests

11.1 Tests unitaires (Vitest)

Fichiers de tests dans **tests/** à côté des composants :

Fichier	Composant testé
AdvancedKpiDashboard.spec.ts	Dashboard KPI
ChartWidget.spec.ts	Widget graphique
KpiStatCard.spec.ts	Carte KPI
DocumentUpload.spec.ts	Upload document
RiskHeatmapInteractive.spec.ts	Heatmap risques
RiskMatrix.spec.ts	Matrice risques
TurtleDiagramEditor.spec.ts	Éditeur diagramme tortue
ConsentBanner.spec.ts	Bannière consentement
GlobalSearchBar.spec.ts	Barre recherche globale
useAuth.spec.ts	Composable auth
auth.spec.ts	Store auth
risks.spec.ts	Store risques
pdfExportService.spec.ts	Service export PDF

11.2 Tests E2E (Cypress)

Répertoire : **cypress/e2e/**

Configuration : **cypress.config.ts**

12. Configuration Vite

Fichier : **vite.config.mts**

Plugins utilisés :

- **@vitejs/plugin-vue** — support Vue SFC
- **vite-plugin-vuetify** — tree-shaking Vuetify
- **unplugin-vue-router** — routes typées
- **unplugin-auto-import** — imports automatiques Vue/Pinia
- **unplugin-vue-components** — enregistrement automatique composants
- **vite-plugin-vue-layouts-next** — layouts automatiques
- **unplugin-fonts** — polices (Roboto)

Alias : **@** → **src/**

Build : **--max-old-space-size=6144** (projets volumineux)